AgenticWorkflow.LangGraph: project guide

Oct 27, 2026 · @Jitendra

Overview

AgenticWorkflow.LangGraph turns a written requirement into reviewed, tested .NET code through five agents, with a person approving every stage. Today the agents are fakes; the workflow around them (pauses, approvals, limits, persistence) is real and tested. Real Claude Agent SDK agents come in Milestone 2.

Stack: LangGraph (workflow and pauses), Claude Agent SDK (the agents, from Milestone 2), FastAPI (the API), Postgres in Docker (runs, saved workflow state, audit trail).

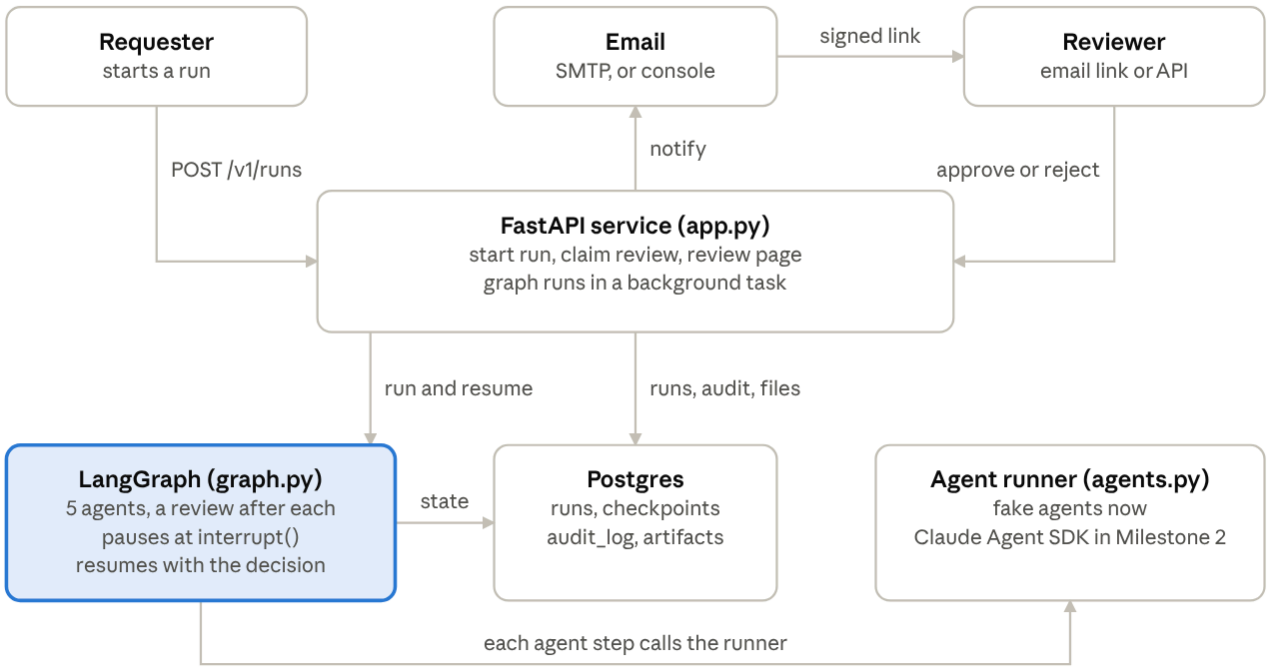
The five stages: spec analyst, designer, implementer, test engineer, reviewer. After each one the run pauses and emails a signed link. Approve moves on. Reject sends the run back to the same agent with the reviewer's comment, up to 3 attempts.

How to describe it honestly: a project built to learn and demonstrate durable human-in-the-loop agent workflows. It runs locally and has not served real users. What it does show: pause and resume across restarts, exactly-one-wins approvals, signed expiring review links, token budgets, loop limits, and protection against stale approvals.

Architecture

The API creates a run, the graph executes it in the background, and after every agent the run pauses until a person decides. Everything needed to resume lives in Postgres, so the server can stop at any point.

A run pauses in the graph and resumes from Postgres after each review



architecture · 7 components, 8 connections

Read it from the top: the requester starts a run in the API, the graph (highlighted) runs the agents and saves its state in Postgres, and each pause sends the reviewer a signed link whose decision comes back through the API.

Status

Milestones 0 and 1 work and pass all 20 automated checks on the author's Windows machine; real agents, the audit and artifact tables, and everything after Milestone 1 are not built yet.

Milestone	Status	What it covers
0. Run what exists	Done	Docker Postgres, FastAPI service, saved state across restarts, email review link
1. Pipeline with fake agents	Mostly done	Five stages with a review after each, reject loop with comment, handoff summaries, attempt and token limits, stale-approval protection
1b. Audit and artifact tables	Next	`audit_log` and `artifacts` tables, files shown on the review page
2. First real agent	Not started	Spec analyst on the Claude Agent SDK, isolated workspace, caps

Milestone	Status	What it covers
3. Remaining agents	Not started	Designer, implementer, test engineer, reviewer, one at a time
4. Hardening	Not started	Auth, Idempotency-Key, timeouts, CI, Dockerfile
5. Polish	Not started	README, demo recording, tracing, cost report

Tested: 9 checks on the graph alone (approve all, reject loop, max rejects, budgets, handoff, retries, worst case, diagram) and 11 on the API with Postgres (validation, full run, reject, stale links, email flow, mail failure, restart in the middle of a run, race). The stale-approval check was proven by removing the protection in a copy and watching the test fail.

Not verified: real SMTP sending, real Agent SDK calls, running with more than one server process.

Code walkthrough

The project is ten small files in `agent_api/`; the workflow lives in `graph.py` and the agents sit behind one function in `agents.py`.

File	Job	Key parts
`agents.py`	The five stages and the agent interface	`Stage`, `STAGES`, `AgentRequest` (what an agent may see), `AgentResult`, `fake_runner`
`graph.py`	The LangGraph workflow	`PipelineState`, `make_agent_node`, `make_review_node`, `decide`, `finish_reason`, `build_graph`
`app.py`	FastAPI service	`lifespan` (pool, checkpointer, graph), `start_run`, `_execute`, `submit_review`, review page routes
`db.py`	The `runs` table	`claim_for_review` (atomic, gate-aware), `mark_*`, `fail_interrupted_runs`
`notifier.py`	Email and signed links	`make_token`, `read_token`, `send_email`, `notify_review_needed`
`run.py`	Starts the server	Switches Windows to the selector event loop psycopg needs

File	Job	Key parts
`docker-compose.yml`	Postgres 17	Port 5432 on localhost, named volume, healthcheck
`smoke_test.py`	20 automated checks	Part A graph only, part B API with Postgres
`requirements.txt`	Dependencies	FastAPI, uvicorn, LangGraph, langgraph-checkpoint-postgres, psycopg, httpx, python-multipart
`PROJECT_PLAN.md`	Short plan in the repo	Milestones, controls, resume bullets

One run, in order. `POST /v1/runs` inserts a `runs` row and returns 202. `start_run` launches `_execute` in the background, which calls `graph.ainvoke`. The first agent runs, its review node calls `interrupt()`, and the graph pauses with its state saved. `_execute` marks the run `WAITING_REVIEW` and sends the email. A decision calls `submit_review`, which claims the review in the database and resumes the graph with `Command(resume=...)`. After the fifth approval the `finish` node sets the outcome and the run is `COMPLETED`.

Key design decisions

Each decision below exists to bound what can go wrong, and each has a reason you can say in an interview.

Decision	Why
The agent is a swappable function (`AgentRunner`)	The graph never knows if agents are fake or real, so Milestone 2 changes one file and tests can inject broken or slow agents
A fixed graph with one node per stage, built by a factory	The process is known, so a fixed graph is easier to test and bound than a free-roaming agent; the diagram shows every stage
A review gate after every stage	A person catches a bad spec before it becomes bad code; cost of an early reject is small
Reject returns to the same agent with the comment	The agent gets exactly what to fix; attempts are capped at 3 per stage

Decision	Why
Handoff is the previous stage's short summary, capped at 600 characters in code	Keeps each agent's context small; the limit is enforced by code, not by asking the model
`drafts` and `approved` are separate in the state	Only approved work reaches the next stage; rejected drafts never flow forward
Workflow state in Postgres (LangGraph checkpointer), run bookkeeping in `runs`	The run survives restarts; two tables answer two different questions
`claim_for_review` is one SQL statement	Check and change are atomic, so exactly one of ten simultaneous approvals wins, across several server processes too
Every review has a gate id such as `designer:2`, in the link and the SQL	A late click or old email link cannot approve a later review of the same run
GET shows the review page, only POST decides	Mail scanners and link previews open links automatically and must not approve by accident
Mail is sent after the status changes, and a mail failure never fails the run	A fast click never meets a 409, and an SMTP outage leaves the run waiting, not broken
Side effects are outside the review node	`interrupt()` makes the node run again from its first line on resume
Retry policy only for timeouts and connection errors	Retrying a bug repeats the bug and the cost
Token budget checked between stages, plus a recursion limit	The budget stops a spending run; the recursion limit stops a routing bug. A single agent call can overshoot the budget by its own usage, which per-agent caps cover in Milestone 2
One server process, for now	Background tasks live in the process, and startup marks leftover `RUNNING` runs as `FAILED`

Data model

One Postgres database holds three kinds of data: the API's run bookkeeping, LangGraph's saved workflow state, and the audit trail with the files each agent produced.

Table	Owner	Answers	Status
`runs`	`db.py`	What is run X doing right now, and what must the reviewer see?	Built
`checkpoints`, `checkpoint_writes`, `checkpoint_blobs`, `checkpoint_migrations`	LangGraph (`AsyncPostgresSaver`)	Where exactly is the graph, and what does its state hold? Keyed by `thread_id`, which equals `run_id`	Built
`audit_log`	`db.py`	Who decided what, at which review, with which comment	Milestone 1b
`artifacts`	`db.py`	Which files did each attempt of each stage produce	Milestone 1b

`runs`: `run_id` (primary key), `status` (QUEUED, RUNNING, WAITING_REVIEW, COMPLETED, FAILED), `outcome` (APPROVED, REJECTED, STOPPED_BUDGET), `created_at`, `updated_at`, `pending_review` (jsonb: the current review, including its `gate`), `result` (jsonb: the final state), `error`. An index on `status`.

`audit_log` (planned): `id`, `run_id`, `gate` (for example `designer:2`), `stage`, `attempt`, `decision` (approve or reject), `comment`, `via` (`api` or `email_link`), `tokens_used` at the time, `artifact_hashes` (the files the reviewer saw), `created_at`. A unique key on (`run_id`, `gate`) allows one decision per review. The row is written by the same SQL statement that claims the review, so a decision can never be accepted without being recorded.

`artifacts` (planned): `id`, `run_id`, `stage`, `attempt`, `path` (for example `spec.md`), `content`, `sha256`, `bytes`, `created_at`. Every attempt is kept, including rejected ones, so attempt 1 and attempt 2 can be compared and an approval can be tied to the exact file version the reviewer read.

Why two kinds of state: the saved workflow state lets LangGraph resume after a restart, but it is not a good place to query. `runs` and `audit_log` are plain tables you can query with SQL: for example, which stage gets rejected most often, and with which comments.

API and review flow

A client starts a run, polls its status, and answers each review either through the JSON endpoint or through the emailed link. Interactive docs are at `/docs` when the server runs.

Endpoint	Purpose	Success	Errors
`POST /v1/runs`	Start a run. Body: `requirement`, optional `token_budget`	202 + `run_id`	422 invalid body
`GET /v1/runs/{run_id}`	Status plus live state (`state`, `next_node`, `pending_review`)	200	404 unknown run
`POST /v1/runs/{run_id}/review`	Answer the current review. Body: `gate`, `decision` (approve or reject), `comment`	202	404 unknown run; 409 run not waiting, or `gate` is no longer current; 422 invalid body
`GET /review/{token}`	The page opened from the email. Changes nothing	200 page	400 bad or expired link; 409 link is for an earlier review or the run is not waiting
`POST /review/{token}`	The Approve and Reject buttons on that page	200 page	same as above
`GET /health`	Liveness	200	none

Run status: `QUEUED` then `RUNNING` then `WAITING_REVIEW` (once per review, five or more times per run) and finally `COMPLETED` or `FAILED`. A completed run has an outcome: `APPROVED`, `REJECTED` (a stage rejected 3 times) or `STOPPED_BUDGET`.

Review link: a signed token holding the run id, the gate and an expiry (24 hours by default), signed with HMAC-SHA256 using `REVIEW_LINK_SECRET`. It is readable but cannot be changed. Opening it only shows the page. The page shows the stage, the attempt, your earlier comment and the agent's result, with all text escaped. The decision is the POST.

Why `gate` is required in the JSON call: the client copies `pending_review.gate` from the status response. That makes the request mean "approve this review", not "approve whatever is current", so a repeated or late request returns 409 instead of approving the next stage.

Email settings are environment variables: `REVIEWER_EMAIL`, `SMTP_HOST`, `SMTP_PORT`, `SMTP_USER`, `SMTP_PASSWORD`, `MAIL_FROM`, `PUBLIC_BASE_URL`, `REVIEW_LINK_SECRET`, `REVIEW_LINK_TTL_SECONDS`. Without `REVIEWER_EMAIL` the link is printed in the server window. Planned: `GET /v1/runs/{run_id}/audit` and endpoints to read artifacts.

Safety, cost and failure controls

Every control below is enforced in code, not in a prompt, and each one is covered by an automated check.

Risk	Control	Check
An agent loops on a stage	Max 3 attempts per stage; then the run ends as `REJECTED` and names the stage	A3, B4
Spend runs away	`token_budget` per run, checked after every review; ends as `STOPPED_BUDGET`	A4, B4
A routing bug loops	Recursion limit of 40 per call (worst legitimate run is 31 steps)	A8
Context grows stage by stage	Handoff is the previous summary only, capped at 600 characters	A5
A timeout or dropped connection	Retry up to 3 times for those errors only; a retry is not counted as an attempt	A7
A bug in an agent	Not retried; the run is recorded as `FAILED` with the error	A7, B8
A reviewer answers with a typo	Anything but approve or reject raises an error, never counts as approval	A6, B1
Two people approve at once	One SQL statement claims the review; exactly one wins	B11
A late click or old email link	Decisions name their gate; a stale gate returns 409 and changes nothing	B5, B6, B11
A mail scanner opens the link	GET changes nothing; only the POST decides	B6

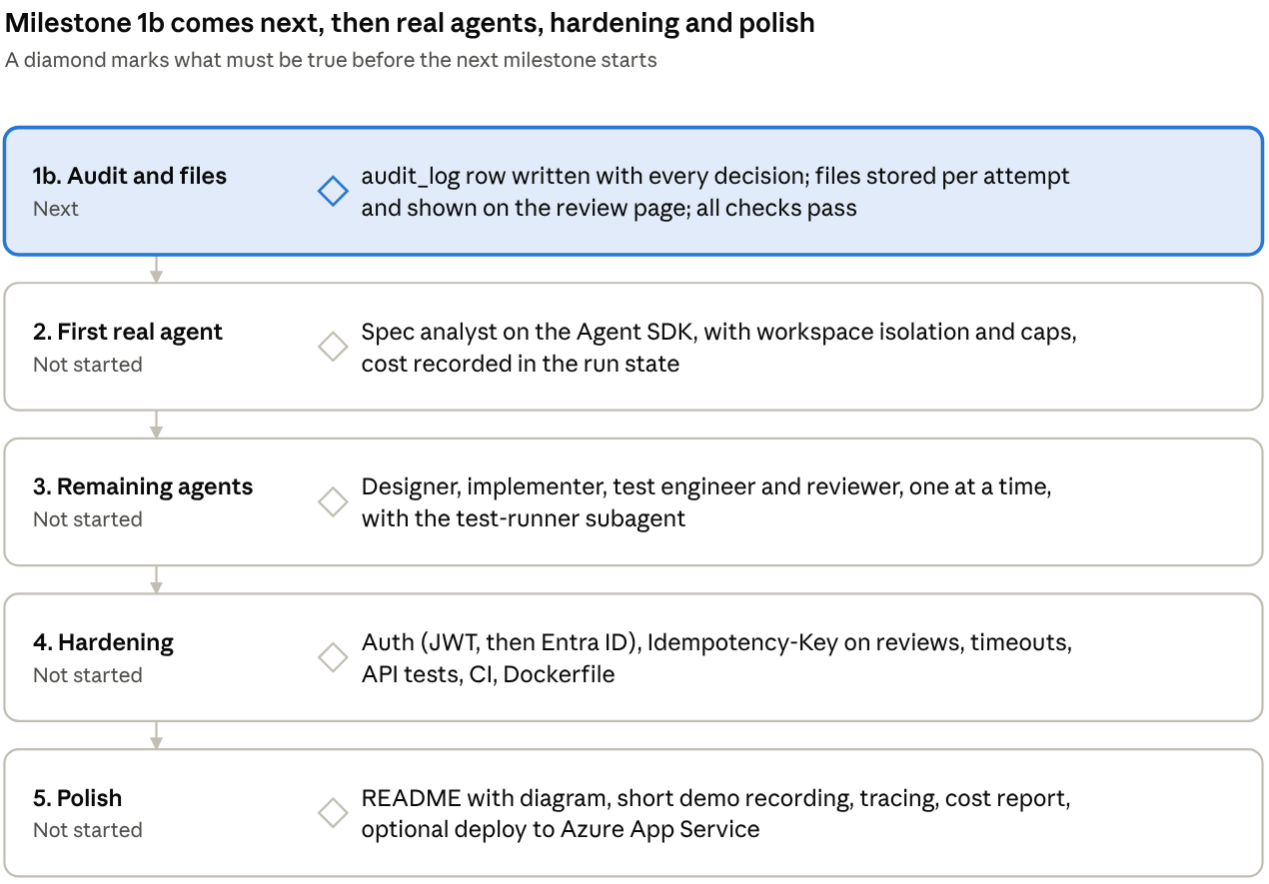
Risk	Control	Check
Model or user text injected into the page	All text is HTML-escaped; checked with a `<script>` payload	B6
Tampered or expired link	HMAC signature and expiry checked; 400	B6
SMTP is down	The run stays `WAITING_REVIEW`; the error is logged	B7
The server dies mid-run	State is in Postgres; runs left `RUNNING` become `FAILED` at startup; waiting runs resume	B9, B10

Known limits: one server process only (background tasks live in the process and startup fails leftover `RUNNING` runs); no authentication on the JSON endpoints yet; the token budget can be overshot by one agent call; no idempotency key on run creation.

Controls that arrive with real agents (Milestone 2): per-agent `max_turns` and `max_budget_usd`, a tool allowlist per stage, an isolated working folder per run, and treating every file an agent reads as untrusted input.

Roadmap

Five milestones remain, and real agents come only after the pipeline around them is complete and tested.



roadmap · 5 milestones, each with a gate

Milestone	Technical detail
1b. Audit and files	The review claim and its `audit_log` row are written by one SQL statement (a CTE), so a decision cannot be accepted without being recorded. Agents return files; the API stores each attempt in `artifacts` with a SHA-256, shows them escaped on the review page, and keeps them out of the email. The audit row records the hashes of the files the reviewer saw. New endpoints read the audit trail and the files

Milestone	Technical detail
2. First real agent	`fake_runner` is replaced by a runner using the Claude Agent SDK `query()` with `ClaudeAgentOptions`: `allowed_tools`, `max_turns`, `max_budget_usd`, `cwd` set to a per-run workspace, and a restrictive `permission_mode`. Cost comes from the result message and feeds `tokens_used`. Files the agent reads count as untrusted input. The sign-in method is still to be decided
3. Remaining agents	One at a time, each with its own tool allowlist (Bash limited to `dotnet` and `git diff`) and caps. The test engineer uses a subagent that returns failures only, so test logs do not enter the main context. Handoff stays summary plus file paths. Needs a small sample .NET repository
4. Hardening	JWT then Entra ID on the endpoints, `Idempotency-Key` on run creation and review, per-agent timeouts, API tests, CI, Dockerfile. `start_run` is the single place to change to move execution to a queue and separate workers, and the startup cleanup then needs to be per worker
5. Polish	README with the diagram, a short demo recording, tracing (OpenTelemetry or LangSmith), a cost report, and an eval run that replays rejected attempts from `audit_log` against a new prompt. Optional deploy to Azure App Service

Run and test locally

You need Docker and Python. On Windows, start the server with `python run.py`, not plain `uvicorn`, because the Postgres driver needs the selector event loop.

1. Start Postgres: `docker compose up -d`, then `docker ps` should show `agent-postgres` as healthy.
2. In `agent_api`, create a fresh virtual environment (they cannot be moved between folders): `python -m venv .venv`, activate it, then `python -m pip install -r requirements.txt`.
3. Run the checks: `python smoke_test.py`. It must end with `ALL CHECKS PASSED`. The two tracebacks for "smtp down" and "boom" are deliberate.

4. Start the server: `python run.py`, then open `http://127.0.0.1:8000/docs`.
5. Start a run with `POST /v1/runs` and `{"requirement": "Add POST /orders"}`. Open the review link printed in the server window after each stage, or answer through `/docs` with the `gate` from `pending_review.gate`.

Without a server or database: `python graph.py` runs the whole pipeline in the console and asks you to approve or reject each stage. `python graph.py --mermaid` prints the workflow diagram as Mermaid text.

Look inside Postgres: in the Docker Desktop Exec tab, `psql -U agent -d agentdb -c "\dt"` lists the tables, and `select run_id, status, outcome from runs order by created_at desc limit 10;` shows recent runs. After a Milestone 1 upgrade, clear old paused runs with `delete from runs;`.

Real email: set `REVIEWER_EMAIL`, `SMTP_HOST`, `SMTP_PORT`, `SMTP_USER`, `SMTP_PASSWORD` and `MAIL_FROM` in the same terminal before `python run.py`. Gmail needs an app password. Keep these out of files and out of Git.

Interview talking points and open decisions

Every answer below is something this project does, so each can be shown in the code and the tests. Say "project" and "local", never "production".

Question	Short answer from this project
How do you pause an agent for a human and resume after a restart?	`interrupt()` in a review node, state in a Postgres checkpointer keyed by run id, resume with `Command(resume=...)`. Check B9 restarts the app between stages
What if two people approve at once?	One SQL `UPDATE ... WHERE status='WAITING_REVIEW' AND gate=...`; the database lets one win. Check B11 runs ten at once
What if someone clicks an old approval link later?	Links and decisions name their gate; a stale gate returns 409. Check B5 and B6
How do you stop an agent loop?	Attempt cap per stage, token budget, recursion limit, all in code
How do you keep each agent's context small?	Handoff is the previous summary only, capped in code; agents read files themselves

Question	Short answer from this project
Why not retry everything?	Retrying a bug repeats the bug and the cost; only timeouts and connection errors are retried
What would change for production?	Auth, idempotency keys, a queue and separate workers, multiple server processes, CI, tracing, evals

Resume bullets, only once true: durable human-in-the-loop pipeline with Postgres-backed checkpoints and gate-bound approvals (true now); five Claude Agent SDK agents with per-agent caps (true after Milestone 3); eval set built from reviewer rejections (true after the audit table and an eval run).

Open decisions:

- [ ] Agent SDK sign-in: the author plans to use a Claude Pro login rather than an API key; whether that is permitted for this use is unverified, so confirm the current terms before building on it.
- [ ] A small sample .NET repository for the agents to work on, for example an empty Orders minimal API.
- [ ] Where artifacts live once agents write real files: the database only, or the run's workspace folder plus a copy in the database.
- [ ] Whether the reviewer may edit a file on the review page, or only approve and reject with a comment.